6.8610 Quiz 2 Study Note

Spring 2026

Aggregated from Lectures 8–15 in `6.8610/.quiz2`

Lightly cross-checked against `6.8610/.quiz2/NLP_notes.pdf`

Cheatsheet Recommendations

- **Use Side A for post-training.** Write the staged pipeline `pretrain -> mid-train -> SFT -> preference learning -> RL`, then the core equations for Bradley-Terry / DPO, the reward-maximization objective, and the PPO / GRPO update rules.
- **Use Side B for retrieval and systems.** Put the retrieval tradeoff table `cross-encoder vs dual-encoder vs late interaction`, the InfoNCE loss, the ColBERT MaxSim score, the RAG factorization, and a tiny ReAct / CodeAct summary.
- **Memorize comparisons, not paper names.** Expect "why is method A better than B here?" more than "what year was paper X?".
- **High-yield formulas to handwrite:**

$$\mathcal{L}_{\text{DPO}}, \qquad J(\theta) = \mathbb{E}[r] - \lambda\,\text{KL}, \qquad \nabla_\theta \mathbb{E}[r] = \mathbb{E}[\nabla_\theta \log p_\theta\, r],$$

$$\mathcal{L}_{\text{InfoNCE}}, \qquad S_{\text{ColBERT}}(q,d) = \sum_i \max_j E_{q_i}^\top E_{d_j},$$

$$p(a \mid q) = \sum_d p(d \mid q) p(a \mid q, d).$$

- **High-yield contrasts to handwrite:** SFT vs ranking losses vs RL; cross-encoder vs dual-encoder vs late interaction; closed-book QA vs RAG vs agents; byte-level vs BPE / WordPiece / Unigram.
- **Always reserve space for failure modes.** This course repeatedly asks not just what works, but what breaks: SFT can teach style over truth, DPO can amplify noisy majority preferences, dense retrievers can fail OOD, RAG can memorize if retrieval is bad, agents suffer context rot, and tokenizer compression does not reliably predict downstream quality.
- **If space is tight, drop examples before dropping objectives.** Keep one anchor example per lecture at most.
- **Do not waste handwritten space on dates, author names, or leaderboard trivia.** Spend it on equations, assumptions, and "what term does what?" explanations.

Course Throughline

- Lectures 8–10 explain how a pretrained next-token predictor becomes a useful assistant: better data staging, supervised demonstrations, preference data, and finally reward optimization with RL.
- Lectures 11–12 explain how models access external knowledge: first by learning retrieval systems, then by combining retrieval with answer generation.
- Lectures 13–14 move from "model as chatbot" to "model as policy inside a system" with tools, long-horizon interaction, prompt optimization, and modular AI programming abstractions.
- Lecture 15 steps back and asks a representation question: before any of the above works, what basic discrete units should text be broken into?
- Repeated theme: the right **inductive bias** often beats naive scaling. This is true for post-training, retrieval structure, agent scaffolding, and tokenization.

Lecture 8: Post-Training

Staged recipe:

pretraining → mid-training → SFT → preference learning / RL.

SFT objective:

$$\max_\theta \sum_{(x,y)\in\mathcal{D}} \log p_\theta(y \mid x),$$

usually with the loss masked on user tokens so the model is supervised mainly on assistant/output tokens.

What mid-training is doing:

- Continue next-token prediction, but on smaller, more curated data: code, books, technical writing, research papers, Q&A, long-context or agent-style traces.
- Early training can use shorter contexts for compute efficiency; later stages can increase context length once the model already has basic language priors.
- High-quality data is scarce, so it is more valuable *after* broad pretraining than before.
- Later batches shape the model more strongly, so you would rather the model forget early boilerplate than later high-value technical data.

What SFT is doing:

- It does *not* replace pretraining. It nudges and extracts capabilities already latent in the pretrained model.
- It teaches instruction following, chat format, refusal style, and response structure.
- Chat templates matter: tokens for system/user/assistant roles are part of the learned behavior.
- If a new environment or tool setup appears later, the hope is that natural-language priors make "post-post-training" relatively cheap.

Why naive SFT is limited:

- It has a labeling bottleneck: you need prompt/desired-answer pairs.
- It often forces one "gold" answer even when many answers would have been fine.
- It can teach the **style** of competence without the deeper process that produced the answer.
- It can worsen hallucination by pushing the model to sound authoritative rather than calibrated.
- "Say I don't know" is often hard to teach from demonstrations alone.

Exam intuition: post-training works partly because pretrained models already contain broad language and world priors; later stages are steering those priors, not starting from scratch.

Lecture 9: Ranking Objectives for Human Feedback

Setup:

$$x = \text{prompt/context}, \qquad y_w = \text{winner}, \qquad y_\ell = \text{loser}.$$

SLiC / margin-ranking idea:

$$m(x, y_w, y_\ell) = \Delta(y_w, y_\ell) + \log p_\theta(y_\ell \mid x) - \log p_\theta(y_w \mid x),$$

$$\mathcal{L}_{\text{SLiC}} = \sum_{x, y_w, y_\ell} \max(0, m(x, y_w, y_\ell)) + \lambda \, \text{KL}(p_\theta \,\|\, p_0).$$

- Δ: required margin; can depend on similarity or task heuristics.
- p_0: reference or base model, often the SFT model.

Bradley-Terry preference model:

$$P(y_w \succ y_\ell \mid x) = \sigma(r(x, y_w) - r(x, y_\ell)).$$

Bradley-Terry negative log-likelihood:

$$\mathcal{L}_{\text{BT}} = -\sum_{x, y_w, y_\ell} \log \sigma(r_\theta(x, y_w) - r_\theta(x, y_\ell)).$$

DPO parameterization:

$$r_\theta(x, y) = \lambda \log \frac{p_\theta(y \mid x)}{p_0(y \mid x)} + \lambda \log Z(x).$$

DPO loss:

$$\mathcal{L}_{\text{DPO}} = -\sum_{x, y_w, y_\ell} \log \sigma\left(\lambda \log \frac{p_\theta(y_w \mid x)}{p_0(y_w \mid x)} - \lambda \log \frac{p_\theta(y_\ell \mid x)}{p_0(y_\ell \mid x)}\right).$$

Unlikelihood for unpaired positive/negative data:

$$\mathcal{L}_{\text{UL}} = -\sum_{(x, y^+)} \log p_\theta(y^+ \mid x) - \sum_{(x, y^-)} \log\left(1 - p_\theta(y^- \mid x)\right).$$

- Ranking losses learn directly from **annotated responses**, not from new model samples.
- Pairwise preference is often easier for humans than writing gold answers from scratch.
- Feedback is especially helpful when a model should abstain or say "I don't know" rather than confidently imitate a bad pattern.
- KL-to-reference prevents the model from satisfying only relative winner/loser orderings while drifting into garbage elsewhere.
- DPO does **not** require a separate reward model; it uses LM probabilities plus a reference model.
- KTO is a later variant inspired by asymmetric gain/loss ideas; the lecture treated it as lower priority than DPO.
- Caveat: a single reward or preference function is a simplification; real users do not all want the same thing.

Exam intuition: DPO is elegant because it turns preference fitting into a classification-style loss while implicitly targeting a reward-regularized optimum.

Lecture 10: Reinforcement Learning Objectives

Why RL at all? With limited preference data, it may be easier to learn **what good behavior looks like** than to directly learn the full policy that produces it.

Reward-regularized objective:

$$J(\theta) = \sum_x \mathbb{E}_{y \sim p_\theta(\cdot \mid x)} \left[r(x, y)\right] - \lambda \, \text{KL}(p_\theta \,\|\, p_0).$$

Reward model:

$$r_\phi(x, y) = w^\top h_\phi(x, y),$$

typically "LM backbone + scalar head", fit with the same Bradley-Terry likelihood:

$$\max_\phi \sum_{x, y_w, y_\ell} \log \sigma(r_\phi(x, y_w) - r_\phi(x, y_\ell)).$$

Vanilla policy-gradient identity:

$$\nabla_\theta \mathbb{E}_{y \sim p_\theta(\cdot \mid x)}[r(x, y)] = \mathbb{E}_{y \sim p_\theta(\cdot \mid x)} \left[\nabla_\theta \log p_\theta(y \mid x) \, r(x, y)\right].$$

Interpretation: sample from the model, score the sample, then do reward-weighted self-fine-tuning. If the reward is 0/1, this becomes "keep the good samples, discard the bad ones".

Folding KL into the reward (useful approximation):

$$r'(x, y) = r(x, y) - \log p_\theta(y \mid x) + \log p_0(y \mid x).$$

Value and advantage:

$$V(x) = \mathbb{E}_{y \sim p_\theta(\cdot \mid x)}[r(x, y)], \qquad A(x, y) = r(x, y) - V(x).$$

Importance ratio for PPO:

$$\rho(x, y) = \frac{p_\theta(y \mid x)}{p_{\theta_{\text{old}}}(y \mid x)}.$$

Clipped ratio:

$$\bar{\rho}(x, y) = \text{clip}(\rho(x, y), 1 - \epsilon, 1 + \epsilon).$$

PPO objective:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_{y \sim p_{\theta_{\text{old}}}} \left[\min(\rho(x, y) A(x, y), \bar{\rho}(x, y) A(x, y))\right].$$

GRPO: for a group of samples $y_1, \ldots, y_n$ from the same prompt,

$$\mu = \text{mean}(r_i), \qquad \sigma = \text{std}(r_i), \qquad A_i = \frac{r_i - \mu}{\sigma},$$

then plug A_i into the same PPO-style clipped objective.

- Vanilla PG is unbiased but high variance and on-policy.
- Baselines do not change the expectation; they reduce variance.
- PPO adds three practical fixes: advantage estimation, sample reuse via importance sampling, and clipping for stability.
- GRPO removes the learned value model and uses within-group reward normalization instead; this has become common in reasoning-model training.
- The professor's key story: learn the reward from scarce human labels, then use extra compute to optimize that reward on **new model outputs** with no additional human annotation.

Exam intuition: vanilla PG is the conceptual core; PPO is the practical workhorse; GRPO is the simpler modern variant when many samples per prompt are available.

Lecture 11: Neural Information Retrieval

Task: given query q, corpus D with N documents, return top-k relevant documents quickly.

Basic retrieval metrics:

$$\text{Success@}\,k = \mathbf{1}[R_k(q) \cap G(q) \neq \emptyset],$$

$$\text{Precision@}\,k = \frac{|R_k(q) \cap G(q)|}{k}, \qquad \text{Recall@}\,k = \frac{|R_k(q) \cap G(q)|}{|G(q)|}.$$

- $R_k(q)$: retrieved top-k set
- $G(q)$: gold relevant set

Key complexity lens:

- Cross-encoder: high quality, but $O(N)$ Transformer calls per query.
- Dual-encoder: $O(1)$ Transformer calls per query, but still $O(N)$ dot products.
- ANN / IVF: heuristic route toward roughly $O(\sqrt{N})$ candidate scoring under balanced clustering assumptions.

Dual-encoder scoring:

$$s(q, d) = f(q)^\top g(d).$$

InfoNCE contrastive loss:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(f(q)^\top g(d^+)/\tau)}{\exp(f(q)^\top g(d^+)/\tau) + \sum_i \exp(f(q)^\top g(d_i^-)/\tau)}.$$

- d^+: positive document
- d_i^-: sampled negatives
- τ: temperature

Late interaction / ColBERT score:

$$S(q, d) = \sum_i \max_j E_{q_i}^\top E_{d_j}.$$

- E_{q_i}: embedding of query token i
- E_{d_j}: embedding of document token j

IVF sketch:

- Cluster document vectors into about $\sqrt{N}$ buckets.
- At query time, compare to $\sqrt{N}$ centroids, then score only docs in the nearest bucket(s), about $O(\sqrt{N})$ docs if clusters are balanced.

- Cross-encoders are great rerankers, not good first-stage retrievers.
- Hard negatives matter more than random negatives because random negatives quickly become too easy.
- Single-vector dense retrieval often struggles most on **generalization / OOD robustness**, not just expressivity.
- Late interaction is the "missing middle": independent precomputable encodings plus fine-grained token-level matching.
- ColBERT storage blowup is handled by smaller vectors plus aggressive compression / quantization.
- ANN still works for late interaction because a doc can only score highly if at least one of its token vectors gets close to a query token.

Exam intuition: the design move is repeated decomposition: across documents, then between query and document, then across tokens inside a document.

Lecture 12: Retrieval-Augmented Generation

Closed-book open-domain QA: answer from model parameters alone.

Open-book open-domain QA: store knowledge explicitly in documents and learn to retrieve and use them.

RAG factorization:

$$p(a \mid q) = \sum_{d \in \mathcal{D}} p_\eta(d \mid q)\, p_\theta(a \mid q, d).$$

In practice, this is approximated over top-K retrieved documents.

Reader styles:

- Extractive reader: predict an answer span from retrieved text.
- Generative reader: decoder generates the answer conditioned on retrieved text; can paraphrase or combine evidence across documents.

Why closed-book QA is not enough:

- Parametric knowledge is static.
- Provenance is opaque.
- Private or fresh corpora may never appear in pretraining.
- Using parameters to memorize huge long-tail fact sets may be inefficient.

Retriever-supervision problem:

- Often the dataset gives only (q, a), not gold relevant documents.
- Heuristics include: use an existing retriever, ask humans for relevance, or mark answer-containing documents as pseudo-positives.
- But search relevance and answer usefulness are related, not identical.

Why latent-document marginalization was attractive:

- Better credit assignment than RL when the retriever found the right doc but the reader still failed.
- But it is hard to initialize, can push the reader to memorize answers if top-K is junk early on, and makes index refresh expensive.

Fusion-in-Decoder (FiD):

- Encode each retrieved passage separately.
- Let the decoder attend over all passage encodings together.
- This buys long effective context without one giant quadratic joint encoder.

Multi-hop / deep research:

- Earlier benchmarks: multi-hop factoid QA.
- Modern framing: "deep research" systems that iteratively search, browse, condense evidence, and answer.
- Baleen-style systems combat combinatorial explosion via iterative retrieval plus evidence condensation / compaction.
- A major lecture pivot: maybe retrieval is best viewed as a **tool** the LM learns to use in text space, rather than something always trained end-to-end with the generator.

Exam intuition: RAG is fundamentally about **credit assignment** and **where knowledge should live**: in weights, in retrieved context, or in a policy that knows how to query tools.

Lecture 13: Agents and Inference Scaling

Inference-time scaling idea: some tasks improve if the model is allowed to think longer, sample more trajectories, or use tools before committing to a final answer.

Pass@1 vs pass@k:

- **pass@1**: expected accuracy from a single rollout.
- **pass@k**: success if at least one of k rollouts solves the task.

Agent loop abstraction:

$$s_t \xrightarrow{\text{LM policy}} a_t \xrightarrow{\text{environment/tool}} o_{t+1} \to s_{t+1}.$$

Tool use in token space:

- Model emits special tool-call syntax.
- Generation is interrupted.
- External system executes the tool.
- Observation is appended to context.
- Generation resumes.

ReAct vs CodeAct:

- **ReAct**: interleave "thought" tokens with explicit actions; reasoning can be a no-op action that buys more inference-time compute.
- **CodeAct**: take actions by writing code; much more general, but more dangerous and harder to control.

What an agent is in this lecture:

- Not a sharply defined object category.
- A useful abstraction for systems that iteratively pursue goals inside environments.
- Strong post-trained LMs can already look agentic if tools and observations are expressed in language.

Long-horizon problems:

- Large environments, especially codebases.
- Persistent state changes.
- Long contexts cause cost and context rot.
- Context condensation / compaction keeps the active context small while preserving enough history to continue.

Post-training for agent behavior:

- Mid-train heavily on high-quality code and tool-use traces.
- Train across many environments, not just one.
- Use RL on full trajectories when rewards are sparse and long-horizon.
- An agent must stay anchored to the **goal**, not just predict locally likely next tokens.

Exam intuition: agents generalize RAG. Instead of hard-coding all outside computation, we give the model some control over when and how to invoke tools.

Lecture 14: Language Model Programming

Main shift:

$$\text{ad-hoc prompt} \to \text{reliable AI system}.$$

- A one-off chatbot interaction is not the same problem as engineering a maintainable, transparent, reliable system that solves the same kind of task over and over.
- Symbolic code is useful because it gives reusable modules, explicit dataflow, and clearer contracts.
- Natural language is still necessary because many requirements are easiest to specify in English rather than in labels alone.

Central abstraction: natural-language signatures.

- Example mental model: `email thread -> events, action items`.
- The point is to separate **what** the module should do from **how** a particular model or inference pipeline currently does it.
- This localizes ambiguity inside module boundaries instead of smearing it across one giant fragile prompt.

Three ingredients of an AI program:

- **Code / modular structure:** how modules compose.
- **Natural-language specs:** task definitions and output contracts.
- **Data / feedback:** edge cases, rewards, and examples of where the first two are insufficient.

Compile the intent into optimized artifacts:

- optimize prompts,
- optimize weights with RL or fine-tuning,
- optimize the pipeline / inference strategy itself.

Prompt optimization themes:

- Bootstrap useful trajectories and search for prompts that make them reusable.
- MIPRO: smarter search over prompt variants instead of naive random search.
- TextGrad / GEPA style methods: use an LM to reflect on failures, propose textual fixes, and iteratively evolve prompts.
- Prompt optimization can be far more sample-efficient than policy-gradient RL when the model is already strong enough to reason about what went wrong.

But prompt optimization is limited:

- It depends on the model already being capable of following the improved instructions.
- It cannot inject missing domain knowledge the model simply does not have.
- Hence prompt optimization and RL / fine-tuning are complements, not substitutes.

Programming abstraction message:

- We should not hard-wire every model-specific prompting trick into application logic.
- Inference strategies like ReAct should be abstracted so they can compose over arbitrary signatures and be learned / optimized too.
- A good abstraction hides model-specific plumbing, just like PyTorch hides low-level tensor kernels.

Exam intuition: this lecture is about moving from "prompt engineering hacks" to a declarative view where AI systems are specified at a higher level and then compiled down into prompts, weights, and pipelines.

Lecture 15: Tokenization

Representation problem: unlike images or audio, text does not come with a natural low-level numeric representation whose distances already mean something useful.

Byte-level tokenization:

- Finite vocabulary, so no OOV problem.
- But sequences become long.
- Byte values are arbitrary with respect to meaning.
- Byte-level generation can drift into gibberish because bytes are not word-like units.

Word-level tokenization:

- Shorter sequences and more intuitive units.
- But morphology, unseen words, multilingual variation, and even the notion of "word" itself make it brittle.

Idealized tokenizer objective:

$$\min_{\Sigma, f} \sum_i |f(x^{(i)})| \qquad \text{subject to } |\Sigma| = V.$$

- Σ: token vocabulary
- f: tokenization function
- V: target vocabulary size

Why this is not solved exactly:

- A perfect tokenizer would balance compactness and expressivity.
- But globally optimal tokenization under natural formulations is intractable, so practical tokenizers use greedy approximations.

Byte-Pair Encoding (BPE):

- Start from bytes / characters.
- Repeatedly merge the most frequent adjacent pair.
- Store the merge history M.
- At test time, apply those learned merges to new text.

BPE lesson from the lecture:

- BPE is a compromise: words are too coarse, bytes are too fine.
- It often compresses token count well, but the learned tokens are not guaranteed to be linguistically satisfying.
- Greedy longest-available segmentation tends to work well at test time.

Pretokenization:

- Hand-built guardrails often stop merges across spaces, punctuation, and digit chunks.
- This is not elegant, but it is practical and important.
- Real production tokenizers use heavy regex-based pretokenization.

Other classics:

- **WordPiece**: BPE-like but merge choice is likelihood / PMI flavored rather than raw count based.
- **Unigram LM**: start with a large vocabulary and prune the least useful tokens.
- **SentencePiece**: framework that supports BPE and Unigram, often useful when whitespace assumptions are undesirable.

What the later "advances" section argued:

- Compression alone does **not** predict downstream performance well.
- Vocabulary size effects can be surprisingly weak over a normal range.
- Pretokenization choices matter a lot.
- No tokenizer clearly dominates all others.
- Byte-level / tokenizer-free approaches are becoming more competitive again.
- Special tokens for system/user/assistant roles are inserted during instruction tuning and are not split into smaller subwords.

Exam intuition: tokenization is a representation choice, not a solved preprocessing detail. The lecture's conclusion is basically "we still do not know what the perfect tokenizer should optimize".

Likely Quiz Habits

- Be able to write each objective and explain what every term is doing in plain English.
- Expect compare/contrast questions framed around sample efficiency, latency, credit assignment, or inductive bias.
- When asked "why does this help?", answer with the bottleneck it addresses: better supervision, better reward credit, better retrieval structure, better tool use, or better representation.
- For retrieval and agents, mention systems constraints alongside model quality. This part of the course is explicitly about quality *and* latency / environment interaction.
- For tokenization, do not overclaim. The lecture repeatedly emphasized uncertainty: good compression is not the same thing as a universally good tokenizer.

Practice Questions and Solutions

Lecture 10 Practice Question (Fall 2024 6.861)

Question.

Consider a simplified language model trained using Reinforcement Learning with Human Feedback (RLHF). We train the model on only one input prompt: $x =$ "What is 6.861?".

The model generates only two possible responses:

1. $y_1 =$ "6.861 is MIT's natural language processing course."
2. $y_2 =$ "6.861 is 13.722 divided by 2."

A reward model assigns scores based on how much each response aligns with human preferences. In this case, the first response is more desired, so the following scores are assigned:

$$r_\phi(x, y_1) = 1, \qquad r_\phi(x, y_2) = 0.$$

The original version of the model, trained through supervised fine-tuning (SFT), considered both responses equally valid, so it assigns the following probabilities:

$$P_{\theta_{\text{SFT}}}(y_1 \mid x) = 0.5, \qquad P_{\theta_{\text{SFT}}}(y_2 \mid x) = 0.5.$$

Recall that the objective of RLHF is to find LM with parameters θ that maximizes the following objective:

$$J(\theta) = \mathbb{E}_{x\sim\mathcal{I}}\left[\mathbb{E}_{y\sim P_\theta(\cdot|x)}\left[r_\phi(x,y) - \beta \log \frac{P_\theta(y \mid x)}{P_{\theta_{\text{SFT}}}(y \mid x)}\right]\right]$$

where $\mathcal{I}$ is some fixed distribution. In this case, $\mathcal{I}$ places all its mass on the single prompt x given at the beginning of this problem.

Your goal is to solve this objective, i.e., find the probabilities $P_\theta(y_1 \mid x)$ and $P_\theta(y_2 \mid x)$ that maxi- mize the objective function.

(a) (2 points) Let $p_1 = P_\theta(y_1 \mid x)$ and $p_2 = P_\theta(y_2 \mid x)$. Write the objective $J(\theta)$ in terms of p_1, p_2, and β.
(b) (1 point) Let $\beta = 1$. Express $J(\theta)$ as a function of p_1 only.
(c) (3 points) Find the value of p_1 that maximizes $J(\theta)$ (for $\beta = 1$).

Solution.

(a) Since there is only one input x, the outer expectation simplifies to evaluating at x. The inner expectation over y becomes a sum over the two possible responses y_1 and y_2:

$$\begin{aligned} J(\theta) &= \mathbb{E}_{y\sim P_\theta(\cdot|x)}\left[r_\phi(x,y) - \beta \log \frac{P_\theta(y \mid x)}{P_{\theta_{\text{SFT}}}(y \mid x)}\right] \\ &= \sum_y P_\theta(y \mid x)\left[r_\phi(x,y) - \beta \log \frac{P_\theta(y \mid x)}{P_{\theta_{\text{SFT}}}(y \mid x)}\right]. \end{aligned}$$

Substituting the known values:

$$\begin{aligned} J(\theta) &= p_1\left[1 - \beta \log \frac{p_1}{0.5}\right] + p_2\left[0 - \beta \log \frac{p_2}{0.5}\right] \\ &= p_1(1 - \beta(\ln p_1 - \ln 0.5)) - \beta p_2(\ln p_2 - \ln 0.5) \\ &= p_1 - \beta p_1 \ln p_1 + \beta p_1 \ln 0.5 - \beta p_2 \ln p_2 + \beta p_2 \ln 0.5 \\ &= p_1 - \beta(p_1 \ln p_1 + p_2 \ln p_2) + \beta(p_1 + p_2)\ln 0.5. \end{aligned}$$

(b) Since $p_2 = 1 - p_1$, we substitute into $J(\theta)$:

$$\begin{aligned} J(\theta) &= p_1 - (p_1 \ln p_1 + (1-p_1)\ln(1-p_1)) + \ln 0.5 \\ &= p_1 - p_1 \ln p_1 - (1-p_1)\ln(1-p_1) + \ln 0.5. \end{aligned}$$

(c) We first compute the derivative of J with respect to p_1:

$$\begin{aligned} \frac{dJ}{dp_1} &= 1 - (\ln p_1 + 1) - (-\ln(1-p_1) - 1) \\ &= 1 - \ln p_1 - 1 + \ln(1-p_1) + 1 \\ &= -\ln p_1 + \ln(1-p_1) + 1. \end{aligned}$$

We can then set the derivative to 0 and simplify:

$$\begin{aligned} -\ln p_1 + \ln(1-p_1) + 1 &= 0 \\ \ln p_1 - \ln(1-p_1) &= 1 \\ \ln\left(\frac{p_1}{1-p_1}\right) &= 1 \\ \frac{p_1}{1-p_1} &= e \\ p_1 &= \frac{e}{e+1}. \end{aligned}$$

Lecture 8 Practice Question (Spring 2026 Quiz 1)

Question.

Which of the following statements about language model pretraining are true? Select all that apply.

(a) An autoregressive language model factorizes the joint probability as

$$P(x_1, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_{<t}),$$

and can therefore compute the exact probability of any text sequence.

(b) In Supervised Fine-Tuning (SFT), the training loss

$$L_{\text{SFT}}(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \log p_\theta(y_t^* \mid x, y_{<t}^*)$$

is typically computed on both prompts x and continuations y^*.

(c) Consider a masked language model that predicts a single given complete left and right contexts (i.e. to predict $p(x_i \mid x_1, \ldots, x_{i-1}, x_{i+1}, \ldots x_n)$. Models pretrained for this masked objective are functionally equivalent to ordinary (autoregressive, "decoder only" LMs): both can directly compute the joint probability $P(x_1, \ldots, x_n)$.

(d) Lower perplexity on a held-out test set guarantees better performance on all downstream NLP tasks such as text classification, question answering, and summarization.

Solution. (a) ONLY.

Explanation. Choice (b) is false because in modern chat SFT, the loss is usually masked on prompt / user tokens and computed mainly on assistant continuations.

Lecture 15 Practice Question 1 (6.461 Fall 2025)

Question.

Determine whether each statement below is true or false.

1. There are no out-of-vocabulary items in a corpus tokenized with BPE.
2. Consider this problem: given a corpus C and a limit K on the vocabulary size, how can we choose up to K tokens and segment C into as few tokens as possible? We have a polynomial time algorithm (with respect to K and $|C|$) that solves this problem.
3. Suppose we do BPE with pretokenization, making sure merges don't cross word boundaries. Doing BPE enough times will eventually result in a training corpus that has length 1.

Solution.

1. True.
2. False.
3. False.

Explanation.

1. BPE avoids OOV by falling back to the base vocabulary used to build the tokenizer.
2. The underlying "pick up to K tokens and segment the corpus as short as possible" problem is not known to have a polynomial-time exact solution; in the course notes it is treated as intractable / NP-complete.
3. With pretokenization, merges do not cross word boundaries, so repeated merging cannot collapse a multi-pretoken corpus into one giant token.

Lecture 15 Practice Question 2 (6.461 Fall 2025)

Question.

Let $\Sigma_0 = \{a, b, \ldots, y, z, \text{space}\}$ be the base token vocabulary (i.e., only lowercase letters and the space token). Further Σ_k be the BPE tokenization vocabulary after k merge steps, starting with the base vocabulary. We thus have

$$|\Sigma_0| = 27, \qquad |\Sigma_k| = 27 + k.$$

Let $f_k : \Sigma_0^* \to \Sigma_k^*$ be the corresponding tokenization function. Given a corpus of text D let $f_k(D)$ be the tokenized corpus, and further let $|f_k(D)|$ be the length of the corpus after tokenization.

(a) [2 points] Define a metric $C_k(D)$ that captures the total size of the BPE vocabulary and the corpus D under tokenization f_k. Concretely, we have

$$C_k(D) = |\Sigma_k| + |f_k(D)|.$$

Suppose D_{training} is the training dataset on which the BPE vocabulary was learned. Can we ever have the following?

$$C_{k+1}(D_{\text{training}}) > C_k(D_{\text{training}})$$

If yes, provide an example. If not, argue why the above is not possible.

(b) [2 points] Consider the following text corpus D_{training} for training your tokenizer.

`banana banana ba`

(i) Suppose we perform two iterations of BPE on the above corpus, making sure that the byte pairs do not cross word boundaries. What are the new tokens (i.e., byte pairs) that are added? (If there are byte pairs with equal counts, you should take the byte pair that occurs first in the dataset.)

(ii) Find an alternative sequence of two merges such that the length of the tokenized dataset under this tokenization is smaller than the one found via above.

(c) [2 points] Suppose we train two language models. The first language model, p_{char}, is trained over raw character tokens. The second model, p_{bpe}, is trained over a BPE-tokenized corpus with the corresponding tokenization function f. The perplexities of the respective models on a held-out test set D_{test} are found to be:

$$\text{perplexity}_{\text{char}} = \exp\left(-\frac{1}{|D_{\text{test}}|}\log p_{\text{char}}(D_{\text{test}})\right) = \exp(2),$$

$$\text{perplexity}_{\text{bpe}} = \exp\left(-\frac{1}{|f(D_{\text{test}})|}\log p_{\text{bpe}}(f(D_{\text{test}}))\right) = \exp(6).$$

Which is a better language model, assuming $|D_{\text{test}}| = 400$ and $|f(D_{\text{test}})| = 100$?

p_{char} $\quad$ p_{bpe} $\quad$ It is not possible to tell from the above information.

Explain your answer.

Solution.

(a) No. At each iteration, we add at most one item to the vocabulary, and the training set length decreases by at least 1.

Explanation. Since $C_k(D) = |\Sigma_k| + |f_k(D)|$, one merge changes the two pieces by $+1$ and at most -1 or better, so $C_{k+1}(D_{\text{training}}) \leq C_k(D_{\text{training}})$.

(b)(i)

`an`

`ban`

(b)(ii)

`na`

`ba`

Explanation. This is a nice reminder that greedy BPE is only a local heuristic. A merge sequence that is not chosen greedily at step 1 can still produce a shorter corpus after the same total number of merges.

(c) p_{bpe} is better because the negative log-likelihood of the test set, i.e.,

$$\log(\text{perplexity}) \times \text{length of dataset},$$

is lower.

Explanation. Per-token perplexity depends on the tokenizer's unit of prediction, so the fair comparison is total log-probability of the same raw test text after accounting for tokenized length.